Creating and Managing Spring Beans Using XML Configuration

Spring Framework | XML-based bean configuration | Eclipse + Maven

Prerequisite: Create the Maven project

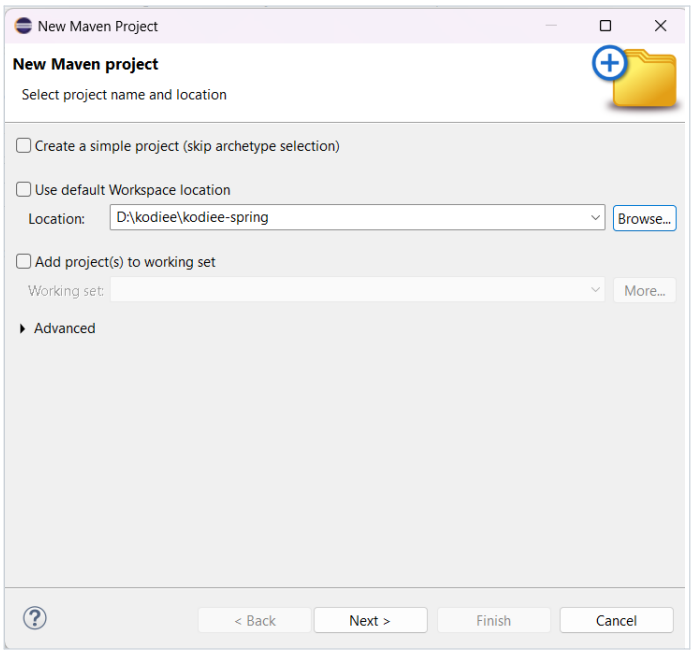
1 In Eclipse: **File >> New >> Maven Project**.

2 Set the workspace location path (see SS1) >> **Next**.

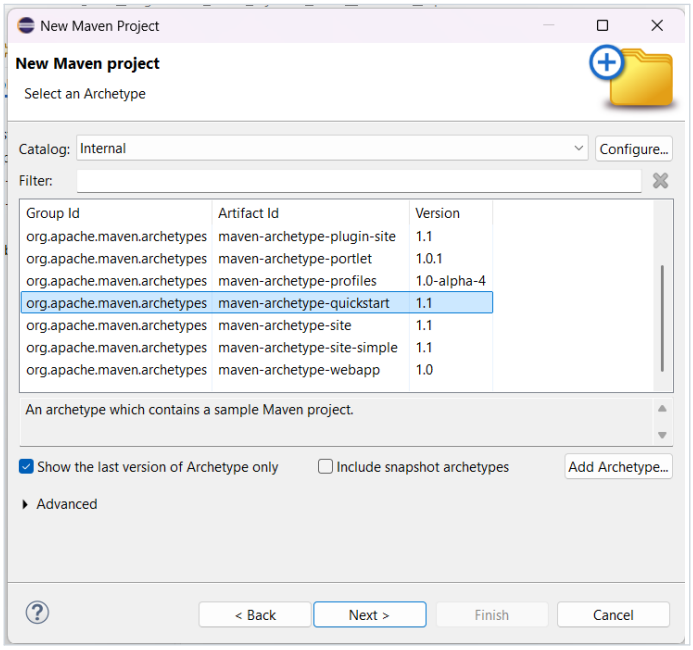
3 Set **Catalog** as **Internal** and select **maven-archetype-quickstart** (see SS2 and SS3) >> **Next**.

4 Set Group Id, Artifact Id, Version and Package (see SS4) >> click **Finish**.

5 Click **Y** on the console when prompted (see SS5).



SS1

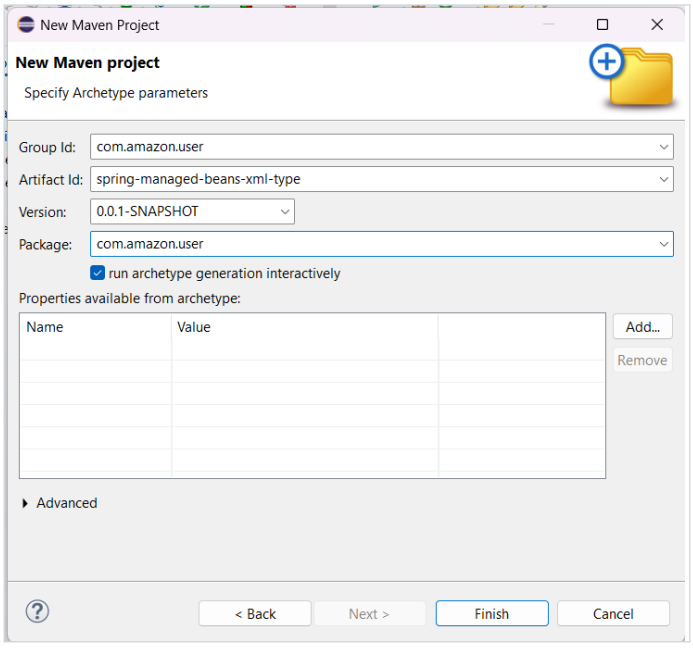


SS2

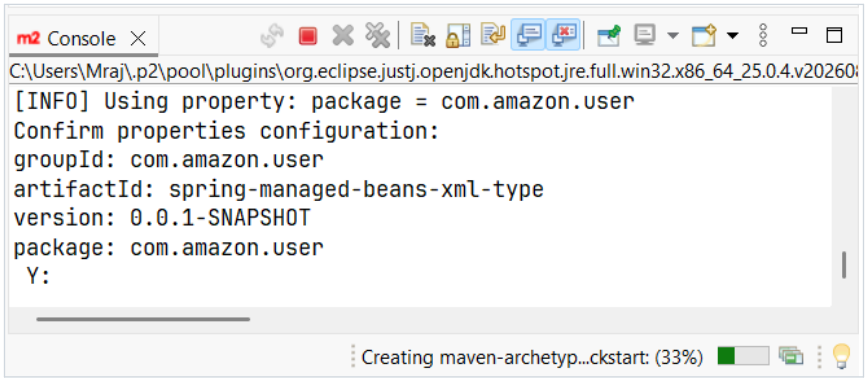


SS3

Group Id	com.amazon.user
Artifact Id	spring-managed-beans-xml-type
Version	0.0.1-SNAPSHOT
Package	com.amazon.user



SS4



SS5

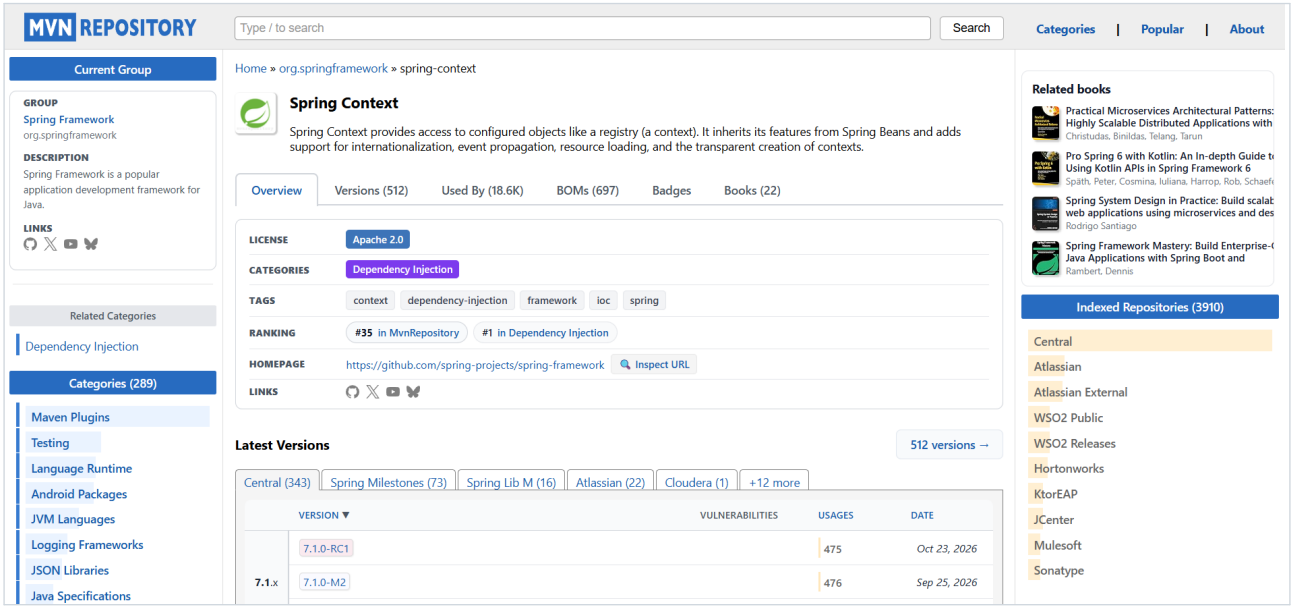
A project will be created with the artifact name we have set, **spring-managed-beans-xml-type**, inside the Package Explorer in Eclipse.

Step 1: Add the Spring Context dependency

Open **pom.xml** and add the dependency of **spring-context** from https://mvnrepository.com/artifact/org.springframework/spring-context.

Always add the dependency of one version behind the latest stable release.

See the dependency (SS6). Added inside pom.xml (SS7).



SS6

spring-managed-beans-xml-type/pom.xml

```
http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.amazon.user</groupId>
    <artifactId>spring-managed-beans-xml-type</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>spring-managed-beans-xml-type</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>

        <!-- Source:
        https://mvnrepository.com/artifact/org.springframework/spring-context -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.2.19</version>
            <scope>compile</scope>
        </dependency>
    </dependencies>
</project>
```

SS7

```
<!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.19</version>
    <scope>compile</scope>
</dependency>
```

Step 2: Create a POJO class

Create one POJO class inside **src/main/java**. Here, **User.java** is created inside the package **com.amazon.user**. Add one default (no-args) constructor and one all-args constructor inside the POJO. The default constructor is used by setter injection (Step 4) and the all-args constructor is used by constructor injection (Step 6).

```
package com.amazon.user;

public class User {

    private int id;
    private String name;
    private String type;
    private String location;

    public User() {
    }

    public User(int id, String name, String type, String location) {
        super();
        this.id = id;
        this.name = name;
        this.type = type;
        this.location = location;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public void displayUserInfo() {
        System.out.println("User [id=" + id + ", name=" + name + ", type=" + type + ", location=" + location + "]");
    }
}
```

Step 3: Create the beans.xml file

Check whether **src/main/resources** is there in your project or not. If present, create one **beans.xml** file there. Otherwise, create the folder by following these steps:

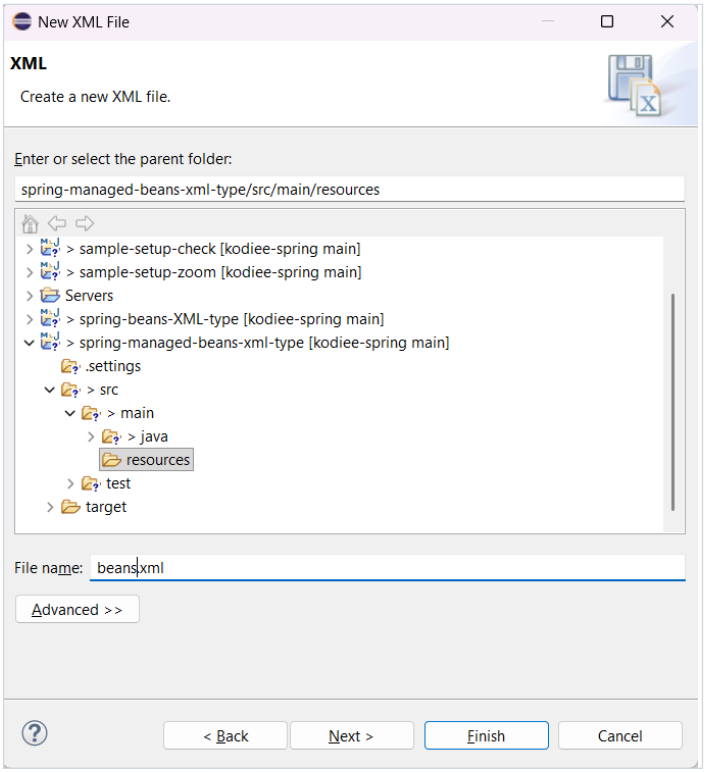
1 Go to the file explorer where your project is located.

2 Go to: `D:\kodiee\kodiee-spring\spring-managed-beans-xml-type\src\main`

3 Create one folder with the name **resources**.

4 Come back to Eclipse and refresh the project.

5 Create **beans.xml** inside **src/main/resources/** (see SS8) >> **Next** >> **Finish**.



SS8

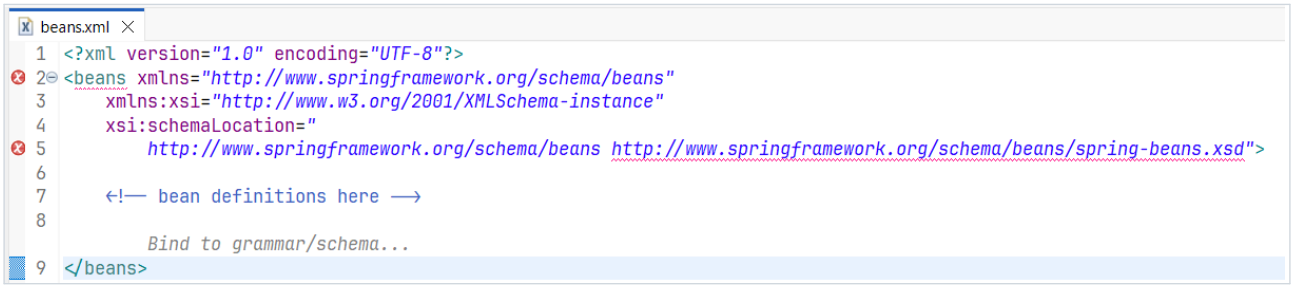
Step 4: Configure beans.xml

Go to this website:
https://docs.spring.io/spring-framework/docs/4.2.x/spring-framework-reference/html/xsd-configuration.html

Copy the blue colour code snippet (SS9). Inside **beans.xml**, clear all the default code and paste it here (SS10).



SS9



SS10

Below the line `<!-- bean definitions here -->`, write our own beans in this way:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- bean definitions here -->

    <bean class="com.amazon.user.User" id="user1">
        <property name="id" value="101"></property>
        <property name="name" value="Alpha"></property>
        <property name="type" value="Non-Prime"></property>
        <property name="location" value="BLR"></property>
    </bean>

    <bean class="com.amazon.user.User" id="user2">
        <property name="id" value="102"></property>
        <property name="name" value="Beta"></property>
        <property name="type" value="Prime"></property>
        <property name="location" value="HYD"></property>
    </bean>

</beans>
```

> **Setter injection:** here Spring uses the **default constructor** to create the object, then calls the setter methods to fill the values. Each `<property>` tag matches a setter by its `name` attribute.

Step 5: Create the IOC container and use the beans

Go to **src/main/java**, **App.java**, where the `main()` method is present, or create our own Driver class where we can write the `main()` method, and follow these 3 steps:

1 Create IOC Container

2 Request for Bean

3 Use the bean

```
package com.amazon.user;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {
    public static void main( String[] args )
    {
        String configFile = "beans.xml";

        // 1. create IOC Container
        ApplicationContext context = new ClassPathXmlApplicationContext(configFile);

        // 2. Request for Bean
        User user1 = context.getBean("user1", User.class); // 1st way
        User user2 = (User)context.getBean("user2"); // 2nd way (Downcasting)

        // 3. Use the bean
        user1.displayUserInfo();
        user2.displayUserInfo();
    }
}
```

When we run the program written inside main(), we get:

```
User [id=101, name=Alpha, type=Non-Prime, location=BLR]
User [id=102, name=Beta, type=Prime, location=HYD]
```

Step 6: Constructor injection (all-args constructor + constructor-arg)

In Step 4, the values were set through the setters. There is another way to create the same object: Spring calls the **all-args constructor** of **User.java** (SS11) and passes all the values in one go, using `<constructor-arg>` tags instead of `<property>` tags.

```
14    public User(int id, String name, String type, String location) {
15        super();
16        this.id = id;
17        this.name = name;
18        this.type = type;
19        this.location = location;
20    }
```

SS11

Add one more bean, **user3**, inside **beans.xml** (SS12), below **user2**:

```
23    <bean class="com.amazon.user.User" id="user3">
24    <constructor-arg value="103"/>
25    <constructor-arg value="Charlie"/>
26    <constructor-arg value="Prime"/>
27    <constructor-arg value="CHN"/>
28
```

SS12

```
<bean class="com.amazon.user.User" id="user3">
    <constructor-arg value="103" />
    <constructor-arg value="Charlie" />
    <constructor-arg value="Prime" />
    <constructor-arg value="CHN" />
</bean>
```

Here, there is no `name` attribute. Spring matches the values to the constructor parameters in the **same order** in which we write them: **id, name, type, location**. Spring also converts the text `"103"` into an `int` automatically.

Step 7: Match constructor-arg by index and by type

Writing the values in the same order as the constructor works, but we can also tell Spring exactly which parameter a value belongs to. There are two more ways:

Way	Attribute	Bean	How Spring matches the value
By order	(none)	user3	In the order the tags are written
By index	`index`	user4	Using the parameter position, starting from 0
By type	`type`	user5	Using the data type of the parameter

By index. Add one more bean, **user4** (SS13). `index="0"` is the first parameter (id), `index="1"` is the second (name), and so on.

```
30    <bean class="com.amazon.user.User" id="user4">
31        <constructor-arg index="0" value="104"/>
32        <constructor-arg index="1" value="Delta"/>
33        <constructor-arg index="2" value="Non-Prime"/>
34        <constructor-arg index="3" value="BBN"/>
35    </bean>
36
```

SS13

```
<bean class="com.amazon.user.User" id="user4">
    <constructor-arg index="0" value="104" />
    <constructor-arg index="1" value="Delta" />
    <constructor-arg index="2" value="Non-Prime" />
    <constructor-arg index="3" value="BBN" />
</bean>
```

By type. Add one more bean, **user5** (SS14). Here, the `type` attribute carries the data type of the parameter, so a String is written as `java.lang.String`.

```
38    <bean class="com.amazon.user.User" id="user5">
39        <constructor-arg type="int" value="105"/>
40        <constructor-arg type="java.lang.String" value="Epsilon"/>
41        <constructor-arg type="java.lang.String" value="Prime"/>
42        <constructor-arg type="java.lang.String" value="KOL"/>
43    </bean>
```

SS14

```
<bean class="com.amazon.user.User" id="user5">
    <constructor-arg type="int" value="105" />
    <constructor-arg type="java.lang.String" value="Epsilon" />
    <constructor-arg type="java.lang.String" value="Prime" />
    <constructor-arg type="java.lang.String" value="KOL" />
</bean>
```

> **Be careful with type.** In **user5**, three parameters share the type `String` (name, type, location). With `type` alone, Spring can't tell which `String` is which, so it assigns them in the order they appear. Here the values happen to be in the right order, so it works. If two of them were swapped, Spring would not complain, and we would get wrong data silently. `index` does not have this problem, so prefer it when a constructor has more than one parameter of the same type.

After adding **user3**, **user4** and **user5**, the complete **beans.xml** looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- bean definitions here -->

    <!-- Setter injection (default constructor + property) -->
    <bean class="com.amazon.user.User" id="user1">
        <property name="id" value="101"></property>
        <property name="name" value="Alpha"></property>
        <property name="type" value="Non-Prime"></property>
        <property name="location" value="BLR"></property>
    </bean>

    <bean class="com.amazon.user.User" id="user2">
        <property name="id" value="102"></property>
        <property name="name" value="Beta"></property>
        <property name="type" value="Prime"></property>
        <property name="location" value="HYD"></property>
    </bean>

    <!-- Constructor injection (all-args constructor + constructor-arg) -->
    <bean class="com.amazon.user.User" id="user3">
        <constructor-arg value="103" />
        <constructor-arg value="Charlie" />
        <constructor-arg value="Prime" />
        <constructor-arg value="CHN" />
    </bean>

    <bean class="com.amazon.user.User" id="user4">
        <constructor-arg index="0" value="104" />
        <constructor-arg index="1" value="Delta" />
        <constructor-arg index="2" value="Non-Prime" />
        <constructor-arg index="3" value="BBN" />
    </bean>

    <bean class="com.amazon.user.User" id="user5">
        <constructor-arg type="int" value="105" />
        <constructor-arg type="java.lang.String" value="Epsilon" />
        <constructor-arg type="java.lang.String" value="Prime" />
        <constructor-arg type="java.lang.String" value="KOL" />
    </bean>

</beans>
```

Step 8: Update App.java and run

In **App.java**, request the three new beans with `getBean()` and use them the same way:

```
package com.amazon.user;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {
    public static void main( String[] args )
    {
        String configFile = "beans.xml";

        // 1. create IOC container
        ApplicationContext context = new ClassPathXmlApplicationContext(configFile);

        // 2. Request for Bean
        User user1 = context.getBean("user1", User.class); // 1st way
        User user2 = (User)context.getBean("user2"); // 2nd way (Downcasting)
        User user3 = context.getBean("user3", User.class);
        User user4 = context.getBean("user4", User.class);
        User user5 = context.getBean("user5", User.class);

        // 3. Use the Bean
        user1.displayUserInfo();
        user2.displayUserInfo();
        user3.displayUserInfo();
        user4.displayUserInfo();
        user5.displayUserInfo();
    }
}
```

When we run the program, all five beans are created and we get (SS15):

```
User [id=101, name=Alpha, type=Non-Prime, location=BLR]
User [id=102, name=Beta, type=Prime, location=HYD]
User [id=103, name=Charlie, type=Prime, location=CHN]
User [id=104, name=Delta, type=Non-Prime, location=BBN]
User [id=105, name=Epsilon, type=Prime, location=KOL]
```

SS15

Whichever way the values are given, the result is the same kind of **User** object. The difference is only in *how* Spring builds it.

Setter injection vs Constructor injection

Point	Setter injection	Constructor injection
XML tag	`<property name=".." value=".."/>`	`<constructor-arg value=".."/>`
Constructor used	Default (no-args) constructor	All-args (parameterized) constructor
How values are matched	By property name (the setter)	By order, by `index`, or by `type`
How the object is built	Object is created first, then values are set one by one	Object is created with all the values in one go
Setters needed in POJO	Yes	No
Beans in this project	user1, user2	user3 (order), user4 (index), user5 (type)